

Online 3-Dimensional Bin Packing-A DRL Algorithm with the Buffer Zone

알고리즘 개요

: 낭비 공간(wasted space)과 버퍼 존(buffer zone)을 정의한 다음, 온라인 3D 빈 패킹 문제를 해결하기 위한 DRL알고리즘을 제안

문제 정의)

- 하나의 bin(파레트)만 부여
- 오직 '현재 도착'한 품목의 정보만 알 수 있음
- 빈 활용도를 극대화
- 현재 도착한 품목을 bin에 더 이상 실을 수 없을 때 적재 과정 종료

가정)

- 품목, bin 크기 > 0
- 품목은 오직 직교하도록만 배치 > 박스를 비스듬히 틀어서 놓는 것 금지
- 품목은 오직 bin의 위쪽에서 수직 방향으로만 배치 > 박스를 옆에서 밀어 넣거나, 끼워 넣는 동작 금지
→ bin 상태를 각 격자 칸의 현재 높이인 'height map'으로 표현이 가능
- 품목은 수평 방향으로의 회전이 허용(rotate horizontally) $> h$ 는 유지되고 l과 w만 회전 가능(0도, 90도)
- 품목은 적재 후 이동 불가능

제약조건)

- 품목은 bin의 경계를 벗어나서 배치될 수 없음
- 품목끼리 겹칠 수 없음
- 품목 배치는 '물리적 안정성'을 만족해야 함

> 물리적 안정성과 수평 회전의 정의는 문헌[3번 논문]을 참조

- **물리적 안정성**
 - 1) 박스 밑면의 60% 초과가 지지되고 네 모서리 전부가 받쳐질 것
 - 2) 밑면의 80% 초과가 지지되고 네 모서리 중 세 개가 받쳐질 것

3) 밑면의 95%가 지지될 것

>> 이를 통해 얻어진 가능/불가능 영역을 이진 행렬인 feasibility mask Mn에 저장해서 정책 학습의 제약으로 씀

- 수평 회전

1) 박스 하나당 가능한 방향이 원래 방향(0도) + 90도 돌린 방향뿐

>> 방향(0/90도)별 feasibility mask 2개를 만들고 그만큼 행동 공간을 2배로 늘려서 '어디에 놓을지, 어느 방향으로 놓을지'를 함께 출력하도록 학습시킴

> 품목 i에 대한 모든 적재 위치의 실행 가능성 : M^i

$$\frac{\sum_{i=1}^N l_i w_i h_i}{LWH} \rightarrow \max,$$

l_i, w_i, h_i : 품목 i의 길이, 너비, 높이

L, W, H : 빈의 길이, 너비, 높이

알고리즘 상세

DRL 알고리즘은 BPP를 마르코프 결정 과정(MDP)으로 정형화

용어 설명)

t: 단계

s_t : bin state(빈의 적재 현황) + item state(현재 박스 정보)

a_t : 에이전트의 박스 적재 위치(x,y) 결정

$R_t(s_t, a_t)$: 그 행동의 결과로 얻는 점수(채운 부피 - 낭비 공간)

s_{t+1} : 박스를 놓고 나서 갱신된 빈 상태

이 과정을 반복하며

policy $\pi(a_t|s_t, \theta)$

- 현 상태에서 어느 위치에 놓을 확률이 얼마인가를 출력하는 정책 함수의 파라미터를 학습(θ = 신경망의 파라미터)

$$u_t = \max_{\theta} \sum_{t=1}^T \gamma^{t-1} R_t(s_t, a_t),$$

- 전체 적재 과정(t=1부터 T까지)에서 받는 보상의 합을 최대로 만드는 파라미터 θ 를 찾는 목표 함수

- γ (할인 인자, discount factor) : [0,1]인 값으로 매 단계마다 곱해져 나중에 받는 보상일수록 가치를 깎아 내림 → 현재 상태의 올바른 적재 보상이 미래의 올바른 적재 보상보다 더 중요하다고 간주

Action 정의)

Bin의 한 모서리를 원점 (0,0) 설정

박스 배치 : 박스의 front-left-bottom 모서리를 격자의 어느 칸 (x_n, y_n) 에 맞출지

(x_n, y_n) 행동 공간 $A = [(x_0, y_0), \dots, (x_{W-1}, y_{L-1})]$ (size = LxW)

Reward function 정의)

$$R_t(s_t, a_t) = \varphi \times \left(\frac{l_t w_t h_t}{LWH} - \frac{V_{waste}}{LWH} \right),$$

첫째항 = 방금 놓은 박스 부피 / 빈 전체 부피 → 클수록 좋음

둘째항 = 방금 행동으로 못 쓰게 된 빈 공간의 크기 / 빈 전체 부피 → 작을수록 좋음

(Figure30에서의 빨간 점선 영역)

Pi = 보상의 크기를 조절하는 하이퍼 파라미터

Buffer Module 정의)

낭비 공간(wasted space)이 크게 발생하는 품목을 적재하게 된다면, 이는 좋지 못한 적재 결과로 이어질 수 있어 버퍼 공간(buffer space)을 정의

- 제약 사항
 - 버퍼 존에 들어갈 수 있는 품목의 최대 개수를 B로 설정
 - 낭비 공간의 임계값을 δ 로 설정합니다. 발생하는 낭비 공간이 임계값보다 크고 버퍼 공간에 있는 품목의 수가 B보다 적다면 버퍼 존에 저장
 - '선입선출(First-Come-First-Packed, 먼저 들어온 품목을 먼저 적재)' 배치 규칙을 채택 & 다음에 적재할 품목으로 버퍼 존에 **가장 먼저 들어온 품목(→ 개선 point)**과 컨베이어 벨트에 막 도착한 품목 중에서 무작위로 선택
 - **CJ과제 기준 변경 사항)** 현재 과제 특성상, 초기에 설정한 버퍼 개수만큼 비는 순간 바로바로 채우는 방식이라서 초기 설정한 버퍼 개수만큼과 현재 컨베이어에 들어온 박스 중 가장 올리기 적합한 박스를 택하는 방식으로 알고리즘을 수정

CNN 모델 알고리즘

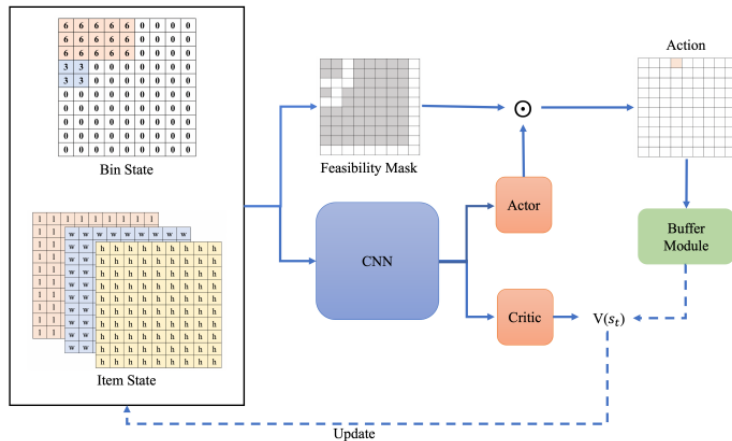


Figure 2. Network framework with the buffer zone

1. 입력

St : Bin State + Item State

- Bin State : bin을 $L \times W$ 격자로 나눠 각 칸에 쌓인 높이를 적은 height map인 H_t 로 표현
- Item State : 현재 박스 크기 $[l, w, h]$ 를 각각 $L \times W$ 격자에 펼친 3채널 텐서

** $L \times W$ 격자에 펼친 3채널 텐서

: CNN모델은 기본적으로 이미지 처리 모델이기 때문에 격자 위치에 쌓인 여러 채널의 값을 묶어서 함께 봄(컬러 사진이 세로x가로x3채널[R,G,B]인 것과 동일 - 세로x가로(= $L \times W$)x3채널(l, w, h))

2.CNN : 특징 추출

: Bin state와 Item State를 합쳐 CNN에 넣어, 의사결정에 쓸 feature를 뽑아냄

ex. 3×3 필터를 특정 격자(3,5)위치에서 주변 3×3 영역의 4개 채널 값을 전부 읽어서 가중합을 계산해서 숫자 1개를 출력.

→ 이를 통해 국소적 패턴 파악(평평, 울퉁불퉁, 박스놓았을 때 단차 발생 여부 등 공간적 관계 포착

→ ‘안정적으로 놓을 자리’는 ‘주변 칸들과의 높이 관계’ 문제라 합성곱이 적절

ex. 주변 가중합으로 중앙이 주변보다 낮으면 음수 출력

ex. 주변 가중합으로 중앙이 주변보다 높으면 양수 출력

ex. 평평하면 0에 가까운 수 출력

→ 이 때, 적절한 가중치로 가중합을 계산하도록, 또 적절한 Actor 결과를 도출하도록 R_t 보상함수(부피 채움 - 낭비 공간) 역전파 전개

3. 출력 : Actor + Critic

- Actor : 각 위치(x,y)에 박스 모서리를 놓을 확률(policy 함를 통해 행동 **at**를 도출하는 역할)
- Critic: 현재 상태가 얼마나 좋은지 $V(st)$ 로 평가, Actor의 행동이 좋았는지 채점하는 역할

4. Actor x Feasibility Mask

: Feasibility Mask는 각 위치에 놓는 게 물리적으로 가능한지를 0/1로 표현한 $L \times W$ 격자

: 이를 Actor의 확률 출력과 곱해서(element-wise), 안정성을 위반하는 위치는 확률을 0으로 만듦

5. Buffer Module

: 선택된 행동을 바로 실행할지, 박스를 버퍼에 빼돌지를 결정

- 판단 기준 ①이 위치에 놓으면 낭비 공간이 임계값을 넘는가? ②버퍼에 자리가 있는가?
- 1&2 모두 충족하면 버퍼에 위치, 아니라면 bin에 위치

6. Update

Critic이 매긴 가치 $V(st)$ 를 바탕으로 보상과 비교해, 네트워크 파라미터 θ, ω 를 갱신

(실제로 받은 보상인 R_t 와 $V(st)$ -추정값을 비교해서 Actor를 갱신

입력 데이터 상세

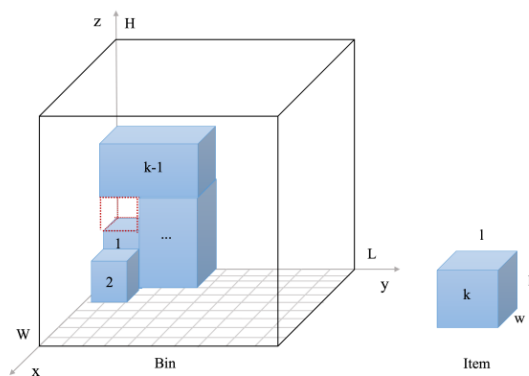


Figure 3. The packing status of the bin and the information about the next item to be packed

- Bin은 $L \times W$ 의 격자로 분할(길이1짜리)
- 그림의 빨간색 점선 > 낭비 공간(wasted space)
- 각 격자 위의 값은 빈 내부에 배치가 끝난 현재 높이(H_t 로 표현)

- 박스 상태=3채널 LxW 텐서 d_t

Ex) $l=3, w=2, h=5, \text{bin}=10 \times 10$

1번 channel = 10x10칸이 전부 숫자 3

2번 channel = 10x10칸이 전부 숫자 2

3번 channel = 10x10칸이 전부 숫자 5

→ 빈 상태와 박스 상태의 크기를 똑같이 맞추기 위해

→ 입력 텐서 $\text{shape}=(10,10,4)$

= channel0: height map / channel1: $l(10 \times 10)$ / channel2: $w(10 \times 10)$ / channel3: $h(10 \times 10)$

용어)

H_n : n번째 박스를 놓아야하는 시점의 bin height map

$[N]^{L \times W}$: $L \times W$ 크기의 격자이고, 각 칸의 값은 N이라는 집합에서 도출

$N \in [0, H]$: 그 칸에 들어갈 수 있는 값의 범위(0=쌓인 것이 없는 상태, H:빈을 꽉 채운 최대 높이)

[모델 학습 결과]

===== [SWEEP 결과 요약] (총점 = 적재율*100 + (20-B)) -20000 + earlystopping=10

===== B= 1 → best total_score = 69.07 (ep:14200)

B= 5 → best total_score = 45.44 (ep:2200)

B=10 → best total_score = 60.94 (ep:8800)

B=20 → best total_score = 38.55 (ep:2200)

>>> 최적 버퍼 크기 B = 1 (총점 69.07)

B= 2 → best total_score = 53.50 (ep 5000)

B= 4 → best total_score = 58.03 (ep 6600)